ChatGPT

Emerging AI-Hardware Accelerators

Introduction and Overview

Modern deep learning models – especially large language models (LLMs) – have insatiable computational demands. Traditional CPUs alone cannot efficiently handle the parallelism required for training and running these models. This need has led to a diverse landscape of **AI hardware accelerators** designed to dramatically speed up deep neural networks while optimizing energy efficiency. This study guide surveys current and upcoming AI chips – from ubiquitous GPUs to specialized TPUs, IPUs, neuromorphic chips, and even emerging optical processors – and examines the architectural innovations that power them. We also overview the software stacks (CUDA, ROCm, XLA, etc.) that interface with these accelerators, and discuss performance-per-watt trade-offs crucial for LLM workloads. A hands-on **Benchmark Lab** is included to compare a 2-billion-parameter Transformer model across different hardware backends. By the end, you should understand the **state of AI accelerators** (circa 2024–2025) and how to leverage them for large-scale AI tasks.

AI Accelerator Landscape: GPUs, TPUs, IPUs, ASICs, Neuromorphic, Optical

Today's AI hardware spans from general-purpose **Graphics Processing Units (GPUs)** to highly specialized accelerators:

- **GPUs – The Workhorse of Deep Learning:** Originally built for graphics, GPUs became the first widely-used AI accelerators when researchers discovered their massively parallel cores excel at matrix and vector math in neural networks [1] [2]. NVIDIA's GPUs (currently ~80% market share in AI datacenters) have evolved rapidly, going from a few dozen cores to thousands of simpler cores optimized for throughput [3]. For example, the 2006 NVIDIA Tesla (G80) GPU introduced a many-core design that doubled performance over its predecessor, and the 2010 Fermi architecture delivered an 8× speedup by further increasing core count and speed [3]. Modern GPUs like NVIDIA's A100/H100 or AMD's MI250 pack specialized Tensor Cores (described later) and use fast on-chip memory caches and high-bandwidth memory (HBM) to feed data to the compute units. GPUs are versatile and excel at both training and inference for a wide range of models. Their main drawbacks are high power consumption and cost: GPUs deliver state-of-the-art performance but **consume a lot of power and are expensive** relative to other solutions [4]. Nonetheless, they remain the default hardware for most AI work due to their maturity and rich software support. Recent trends also include multi-GPU systems and NVLink/NVSwitch interconnects to scale out to multi-node supercomputers for training giant models.

- **TPUs – Google's Tensor Processing Units:** To reduce dependency on GPUs, Google developed custom **Tensor Processing Units (TPUs)**, first deployed internally in 2015–2016. TPU v1 was an inference-focused ASIC for running CNNs and early ML models, soon followed by TPU v2 and v3 for training deep networks with support for floating-point math (bfloat16) and larger memory. TPUs use

a **systolic array** architecture – essentially large 2D arrays of multiply-accumulate units that perform matrix multiplication in a pipeline. For example, TPU v2/v3 contain a 128×128 matrix multiply unit that can sustain high throughput on dense matrix ops. Google reports that its latest **TPU v4** (introduced around 2021) outperforms TPU v3 by 2.1× and improves performance per Watt by 2.7× [5]. A TPU v4 pod (4096 chips) achieves nearly 10× overall speedup versus the older TPU v3 pod, and can train large language models at ~60% of theoretical peak FLOPS [6] [7]. Compared to GPUs, TPUs are more specialized – they lack some of the flexibility (e.g. no fine-grained sparseness or dynamic branching support in earlier versions) but excel at regular dense tensor operations. Google has made TPUs available via its cloud; however, **TPUs require using Google's software stack (XLA and JAX or TensorFlow)** and historically had more limited ecosystem than NVIDIA's CUDA [8]. Even so, when used with well-optimized code (e.g. JAX on TPUs), they can outperform GPUs on many training tasks [8]. Other tech giants have followed suit with their own AI chips – e.g. Amazon's AWS **Trainium** (announced 2020) for training and **Inferentia** for inference, Microsoft's rumored Project Brainwave AI chips, and Meta's in-house accelerator efforts [9] – all aiming for better price/performance than commodity GPUs.

- **IPUs (Intelligence Processing Units) – Graphcore and Others:** Graphcore's IPU is a notable **domain-specific architecture (DSA)** designed specifically for AI. Graphcore's Colossus IPU (Mk1 introduced ~2018, Mk2 "Bow" in 2021) consists of **many light-weight cores with large on-chip memory** (on the order of 900MB to 1GB SRAM per chip) arranged in a mesh [10]. The IPU eschews HBM in favor of cheaper off-chip DRAM plus massive on-chip memory, so that most model data can be kept close to compute [11] [12]. This design reduces the need to constantly fetch from external memory, which is a major source of energy and latency. IPUs excel at fine-grained parallelism (e.g. each core can execute part of a neural network graph in parallel, and synchronization is minimized). **According to Graphcore's own reports,** their IPU systems achieve about **2× the energy efficiency of an NVIDIA Ampere GPU and ~3× that of Google TPU v3** on sustained workloads [12] [13]. IPUs have shown strong performance on Graph Neural Networks, transformers, and other large models that can be partitioned across many cores [14]. Besides Graphcore, other AI ASIC startups include Cerebras (with its *wafer-scale engine*, a single enormous chip containing 850,000 cores and 40GB on-chip memory to train very large models in-memory), SambaNova (reconfigurable dataflow units), Groq, Tenstorrent, and more. Each introduces its own architecture tweaks, but all aim at the same goal: maximize parallel throughput on deep learning operations while minimizing data movement and energy. A general trend is **domain-specific chips** trade off generality for efficiency – they are built to run neural nets orders-of-magnitude faster/watt than CPUs, albeit with less flexibility in programming.

- **Neuromorphic Processors:** Neuromorphic chips depart from the conventional compute paradigm entirely. They are inspired by the brain's architecture, using networks of spiking neurons and synapses to compute in an event-driven manner. Rather than dense floating-point MACs, neuromorphic designs process *spikes* (binary or small analog pulses) and often encode information in timing. Two landmark examples are IBM's **TrueNorth** (2014) and Intel's **Loihi** (2018). IBM TrueNorth, built on 28nm, contains 1 million hardware neurons and 256 million synapses (with 1-bit precision) on a single chip [15]. It is designed as a many-core grid of simple neuron cores operating in parallel with minimal power. Intel Loihi (14nm) implements ~130k neurons and 130 million synapse memory, with on-chip learning rules and a configurable neuron model (Leaky-Integrate-and-Fire, etc.) [15]. **Neuromorphic chips can achieve extraordinary energy efficiency** on spiking neural network workloads – measured in picojoules per spike operation. For example, small research

chips like ODIN and MorphIC demonstrated inference at only ~8–30 pJ per synaptic operation [16], orders of magnitude lower energy per operation than conventional digital accelerators which might spend nanojoules (1e3 pJ) or more per operation. However, neuromorphic hardware currently targets niche workloads (e.g. sensing, sparse pattern recognition) and requires algorithms to be formulated as spiking networks. For standard deep learning (CNNs, Transformers), they are not directly compatible without substantial re-thinking of the models. Thus, neuromorphic processors represent a **long-term, bio-inspired approach** that could yield ultra-efficient AI for certain tasks, but they are not yet drop-in replacements for GPUs/TPUs on LLMs. Research is ongoing into bridging the gap (e.g. converting trained ANNs to SNNs, or using SNNs for energy-efficient inference) [17] [18].

- **Optical and Photonic Accelerators:** An emerging frontier is optical computing for AI. Optical or photonic accelerators use light (photons) instead of electricity to perform calculations, typically leveraging the physics of interference and light modulation to execute matrix multiplications **at extremely high speed and low latency** [19] [20]. Recent advances in silicon photonics and nanophotonics have enabled prototype optical neural network chips. For example, a 2024 Nature paper demonstrated a photonic processor with 16,000 optical components performing 64×64 matrix multiplications at up to 1 GHz and as low as 3 ns latency per operation – achieving two orders of magnitude lower latency than electronic accelerators [21] [22]. The appeal of optical computing lies in parallelism and reduced power: light can carry out multiply-accumulate operations inherently as it propagates through a circuit, and it avoids resistive losses and heat that plague electronic transistors [23] [24]. **Optical Transformers:** Researchers have started investigating running Transformer models on optical hardware. Simulations and early experiments indicate that optical matrix-vector multipliers can run large Transformers with accuracy comparable to digital 8-bit precision, provided enough photons (energy) are used to overcome noise [25] [26]. Excitingly, the *asymptotic* energy efficiency of optical matrix multiplication improves with model size – the energy per MAC can scale as 1/width (due to parallelism), giving optical systems a potential advantage that grows for larger and larger models [27] [28]. One study predicts that well-engineered optical accelerators could achieve **~100× better energy-efficiency** than current electronic processors for today's biggest Transformer models, and in the far future (with trillion-plus parameter models and improved optical tech) possibly **>1000× or even 100,000×** better efficiency [29] [30]. These numbers are speculative but underscore the theoretical promise. However, optical computing is still in its infancy: challenges include the lack of optical memory (conversions to electronic domain incur overhead), analog noise and error accumulation, and difficulties in programming these systems [31] [32]. In summary, **photonic AI accelerators** are a cutting-edge research direction that could revolutionize performance-per-watt in the long term, though for now they remain "early results" in labs [33].

Architectural Primitives: Tensor Cores, Sparsity Engines, Memory & More

Despite their differences, most AI accelerators deploy common **architectural tricks** to achieve high performance and efficiency:

- **Tensor Cores and Matrix Units:** Matrix multiplication is at the heart of neural networks (e.g. the bulk of computation in transformers and CNNs is dense matrix-matrix or matrix-vector multiplies). Accelerator architects therefore include fixed-function hardware to perform these ops extremely fast. NVIDIA introduced *Tensor Cores* in its Volta GPU (2017) – hardware units that multiply small

matrices (e.g. 4×4 or larger) in one go, using low precision like FP16/BF16 or INT8. These tensor cores can achieve **much higher throughput (teraflops)** than general-purpose CUDA cores by performing fused multiply-accumulate on many values simultaneously [34] [35]. In effect, NVIDIA GPUs since Volta operate partly like a "GPU + TPU" hybrid, with each Streaming Multiprocessor embedding matrix units for tensor math [34] [35]. Google's TPUs, as noted, use systolic arrays – functionally similar, a large array of MAC units wired in a grid. Other ASICs also feature matrix engines: e.g. **systolic arrays** (each cell computes and passes data along, as in TPUs), or **SIMD dot product units**. The key idea is to maximize *data reuse on-chip* – feed one piece of data to a unit and use it for many multiply-accumulates to amortize memory cost. Modern tensor cores often support mixed precision (multiply in low precision, accumulate in higher precision) to boost speed while preserving accuracy. For instance, NVIDIA's A100 GPU can deliver up to **312 TFLOPS** of FP16/BF16 throughput thanks to its tensor cores, versus <20 TFLOPS of FP32 on regular cores – a huge leap for deep learning workloads. Dedicated tensor hardware has been so successful that even CPUs now include small matrix units (e.g. Intel DL Boost with AMX). In summary, **matrix/tensor cores** are an architectural staple in AI chips, providing "hardwired" acceleration for the dense linear algebra that dominates deep learning [34] [35].

- **Exploiting Sparsity (Sparsity Engines):** Not all computations in a neural network are truly needed – many weights or activations can be zero, especially in pruned or sparse models. *Sparsity engines* seek to skip doing work on zeros, boosting efficiency. There are two kinds of sparsity: **weight sparsity** (some model weights pruned to zero) and **activation sparsity** (e.g. ReLU sets negatives to zero, or in transformer decoders, many attention outputs can be zeroed by masking). Early sparse accelerators (Eyeriss, SCNN, etc.) targeted convolutional nets by storing data in compressed formats and using hardware to skip zeros [36] [37]. Some only handled one-sided sparsity (either weights or activations) [38], while others like *SCNN* handled **two-sided sparsity** (both weights and activations) for maximal savings [37] [39]. Handling sparsity efficiently is *hard* – irregular data access and load balancing issues arise when different neurons have different numbers of nonzeros [40] [41]. Nonetheless, recent accelerators have integrated sparse support. **NVIDIA Ampere GPUs (2020)** introduced **Sparse Tensor Cores** that can double throughput if weights are pruned 2:4 (50% sparse) in a specific pattern [42]. Essentially, Ampere can skip one out of two weight values, achieving up to 1.5× speedups in practice with only minor accuracy loss from structured pruning [42] [43]. This is a fixed sparsity pattern; unstructured sparsity (random zeros) is harder to exploit on GPU, as current Sparse Tensor Cores do *not* accelerate activation sparsity or irregular sparse matrices [44]. More flexible solutions may involve software and hardware co-design – e.g. selecting block-sparse patterns that hardware can utilize, or even **Mixture-of-Experts (MoE)** models where only a subset of the network "experts" are active per input. Google's TPU v4 includes **SparseCores** – specialized dataflow units dedicated to sparse operations, notably accelerating embedding lookups (which are essentially sparse gather operations) by **5–7× speedup for recommendation models** while using only ~5% of the TPU's die area and power [45] [46]. This illustrates the payoff of specialization: a small slice of silicon can handle sparse tasks far more efficiently than the general matrix units. Going forward, we see research on adaptive sparsity (activations that dynamically turn neurons on/off per input) and **reconfigurable sparse accelerators** that can handle varying sparsity patterns [47] [48]. In summary, **sparsity-aware accelerators** aim to improve the *performance-per-compute* by not computing things that don't need computing. The challenge is doing this without introducing too much complexity or overhead for irregular data.

- **On-Chip Memory and Dataflow Optimization:** A major cost in any AI chip is **memory access** – fetching data from off-chip DRAM is orders of magnitude more energy-expensive and slower than doing arithmetic. Therefore, architectures bring memory *on-chip* (SRAM or eDRAM) and carefully plan data movement. High-end GPUs and TPUs feature large on-chip scratchpads or caches and rely on **High-Bandwidth Memory (HBM)** stacks to supply data at hundreds of GB/s. Some accelerators go further: Graphcore's IPU, as mentioned, integrates ~900MB SRAM on chip so that small models can fit entirely on-chip, avoiding external memory except for initial load [49] [11]. Cerebras's wafer-scale chip similarly has 18 GB SRAM on chip for model parameters. Another approach is **Processing-In-Memory (PIM)** or near-memory computing: embedding computation capabilities directly in the memory arrays. Experimental PIM chips using ReRAM or PCM memory can perform analog matrix-vector multiplications inside memory, dramatically reducing data movement [50] [51]. For example, the *NeuRRAM* and *Hermes* prototypes implement CNNs/RNNs with resistive memory crossbar arrays, achieving thousands of GOPS/W by eliminating most off-chip transfers [52] [53]. Aside from adding more memory, architects optimize the **dataflow**: i.e. the order in which operations access data and how data is reused. They employ techniques like *weight stationarity* (keep weights in place, stream inputs through) or *output stationarity* (accumulate partial sums in place) to minimize reading/writing from DRAM [54] [55]. Figure 2 of the survey illustrates these dataflows [56] [57]. In essence, by **keeping data on-chip as long as possible**, accelerators cut down the "memory wall" penalty. Many ASICs also include multi-level memory hierarchies and **tiling** strategies (splitting computations into blocks that fit in local memory). The result of these primitives is a big improvement in effective throughput per Watt, since external memory access is often the dominant energy cost in deep learning. As one example, by choosing an input-stationary or weight-stationary dataflow, an accelerator can reduce DRAM traffic by orders of magnitude, leading to much higher energy efficiency [58] [59]. Almost all modern AI chips – GPUs, TPUs, FPGAs, etc. – rely on these principles of *on-chip memory, reuse, and dataflow* to hit their performance targets.

- **Precision Reduction (Quantization):** Although not a hardware feature per se, using lower numerical precision has become an architectural and algorithmic staple. Many accelerators support 16-bit, 8-bit, or even lower precision arithmetic which can drastically speed up compute and shrink memory usage. *Half precision* (FP16/BF16) is standard for training large models now, and 8-bit or 4-bit integer is emerging for inference of LLMs. Custom formats like Google's bfloat16 (tolerant of large range) or even *Block Floating Point* and *logarithmic* number systems have been explored [60] [61]. Cutting precision reduces energy per operation and allows more ops in parallel for the same area/power. The trade-off is model accuracy – but research in quantization algorithms and mixed-precision training has shown it's often possible to go to INT8 or lower with negligible accuracy loss, especially for mature models. New hardware (e.g. NVIDIA Hopper) even supports **FP8** training. In summary, by co-designing *numerical formats* with hardware, AI accelerators push more throughput for the same power budget [60] [61]. Extreme cases like binary networks (1-bit weights) yield massive speed-ups but typically incur significant accuracy drops (needing more parameters to compensate) [62]. The sweet spot tends to be 4–8 bits for inference and 8–16 bits for training as of 2024, and hardware is aligning with that.

Software Stacks: CUDA, ROCm, XLA, TVM, JAX, Triton

Hardware advances would be moot without software to actually use them. Several key software frameworks and compilers form the bridge between AI models and accelerator chips:

- **CUDA (NVIDIA's Compute Unified Device Architecture):** CUDA is the fundamental software platform for NVIDIA GPUs. It includes the CUDA C/C++ language extensions and APIs to write kernels that run on GPU, along with drivers, libraries (cuBLAS, cuDNN, etc.), and tooling. Introduced in 2007, CUDA played a huge role in popularizing GPU computing [63]. It abstracts the GPU hardware through a programming model where the developer exposes parallelism and memory hierarchy. Over years, NVIDIA aggressively expanded CUDA with deep learning libraries like cuDNN (for neural network primitives) and TensorRT (for optimized inference). **CUDA's dominance** has become a double-edged sword: it provides excellent performance and a rich ecosystem (over *250 CUDA-accelerated libraries* [64]), but it is proprietary, tying software to NVIDIA hardware [64] [65]. Many researchers and enterprises stick with NVIDIA largely because of CUDA's maturity – it's the "moat" that competitors struggle to overcome [64]. NVIDIA's continual updates (supporting new features like Tensor Cores, sparsity, etc. via CUDA APIs) ensure that software can fully leverage new GPU hardware without requiring low-level changes from users. In summary, **CUDA is the de-facto standard** for GPU programming in AI, and knowledge of CUDA or at least its abstractions (as used in PyTorch/TF backends) is essential for optimizing models on NVIDIA GPUs.

- **ROCm (AMD's Radeon Open Compute):** ROCm is AMD's open-source answer to CUDA. AMD's GPUs historically lagged in software – a major weakness [66]. To address this, AMD made ROCm (pronounced "rock-em"), a platform that includes the HIP programming language (which is similar to CUDA but portable), compilers, drivers, and math libraries for AMD GPUs. ROCm allows running frameworks like TensorFlow and PyTorch on AMD hardware by providing a CUDA-compatible API layer (HIP can compile many CUDA codes to run on AMD). While **ROCm has matured** and AMD even open-sourced its libraries to gain community support [66], it still trails CUDA in terms of ecosystem and polish [66]. However, it has enabled alternatives – for example, the Frontier supercomputer (Oak Ridge National Lab, 2022) uses AMD MI250X GPUs with ROCm for large-scale AI and HPC workloads. For users, the ROCm stack means you can in theory take a CUDA-based ML code and run on AMD with minimal changes, though in practice some ops or kernels might not be as optimized. Efforts like MIGraphX and MIOpen (analogous to cuDNN) are part of ROCm. **Portability** is a theme – frameworks now abstract hardware so that if you use PyTorch on AMD, under the hood it uses ROCm instead of CUDA, but your high-level code is unchanged. ROCm is critical to breaking NVIDIA's monopoly, and with support from the open-source community, its performance and coverage have been improving (AMD GPUs are now viable for many training tasks, and AMD cards often offer more memory at lower cost). In summary, **ROCm provides the software foundation for AMD GPUs** in AI, aiming to offer an open, vendor-neutral alternative to CUDA.

- **XLA (Accelerated Linear Algebra by Google):** XLA is a graph compiler initially developed for TensorFlow, now central to JAX and TPU workflows. XLA takes high-level ML computations (expressed as graphs of ops or as JAX NumPy code) and **generates optimized kernels** targeting various hardware backends (CPU, GPU, and especially Google TPUs). When you run a JAX function, XLA JIT-compiles it to machine code for the chosen device, performing optimizations like operation fusion, layout optimizations, and memory planning. XLA is key to achieving high performance on TPUs – it vectorizes ops to map onto the TPU's systolic array, manages data transfers, etc. Outside of Google

hardware, XLA can also compile to GPUs (there's an XLA:GPU backend that uses CUDA under the hood). For example, the *PyTorch XLA* project allows running PyTorch models on TPUs by converting them to XLA graph computations. In practice, XLA's strength is in optimizing large compute graphs holistically rather than each op in isolation. It can fold sequences of elementwise operations, choose optimal convolution algorithms, etc., often yielding speedups. However, writing custom ops that integrate with XLA can be complex. XLA is also evolving as part of TensorFlow's MLIR-based compiler stack. The bottom line: **XLA is a powerful compiler for ML computations**, crucial for unlocking performance on TPUs and also useful on GPUs for languages like JAX. Its use in JAX has demonstrated significant performance gains over eager execution – e.g. one benchmark found JAX (with XLA) 12% faster than PyTorch on the same GPU for a Transformer training step [67] [68]. This shows how compiler optimizations can rival hand-tuned libraries in some cases.

- **Apache TVM (Tensor Virtual Machine):** TVM is an open-source deep learning compiler stack that aims to provide *performance portability*. Developed by academia (UMass, UW, etc.) and now an Apache project, TVM takes a model from frameworks like PyTorch, TensorFlow, or ONNX, and **compiles it to efficient code for a variety of hardware** – CPUs, GPUs, DSPs, FPGAs, ASICs. It applies both graph-level optimizations (operator fusion, layout changes) and **low-level optimizations with an auto-tuner** that searches for the best way to schedule each operator on the target hardware [69] [70]. A key feature is that TVM can generate code that rivals hand-written kernels. For example, TVM autotuning found optimized kernel implementations that match or exceed cuDNN performance for certain convolutions, and it enabled running models on mobile GPUs and custom accelerators with high efficiency [71] [70]. TVM's success is evident in industry adoption – it's used in AWS (for their Inferentia and Trainium chips), by Qualcomm, and others to deploy models on diverse devices. Using TVM, an engineer can write a model once and have it optimized for, say, an ARM CPU or a Mali GPU or a proprietary NPU, without hand-coding for each. It even integrates a learning-based cost model to explore optimization choices automatically [71]. In summary, **TVM is like a meta-compiler for deep learning**: it strives to make it easy to target new AI hardware and extract good performance without relying solely on vendor libraries. This is increasingly important as the hardware landscape fragments – TVM and similar compiler frameworks (Google MLIR, Facebook Glow, etc.) will ensure software can keep up with new chips.

- **JAX (High-Level Numerical ML Framework):** JAX is a Python library developed by Google that has gained popularity in research for its combination of NumPy-like ease of use with composable function transformations (autodiff, JIT, vectorization). JAX uses XLA under the hood: when you `jit` a function, JAX uses XLA to compile it for the chosen backend (CPU, GPU, or TPU). The result is that JAX can often maximize hardware utilization – for instance, JAX can deploy a training step across multiple TPU cores seamlessly. JAX doesn't have a traditional "graph vs eager" interface like TensorFlow; instead, your Python code is staged to XLA when decorated. It also supports **SPMD parallelism** via `pmap` (parallel map) for multi-device training. In practical terms, **JAX + TPU has proven extremely fast** for many tasks, sometimes outperforming equivalent PyTorch on GPU. As noted, Google's JAX + TPU setup often yields state-of-the-art training throughput (it's heavily used in large model research at Google/DeepMind). JAX's advantages include a very flexible API (you can write things in pure Python+NumPy style and get it compiled) and powerful ** composability **(e.g. you can `vmap` to vectorize any function, or `grad` to get its derivative, and these compose). One caveat is smaller ecosystem (fewer pre-built models than PyTorch, though libraries like Flax and Haiku fill the gap). Nonetheless, JAX is now a viable alternative for cutting-edge research. It has also been demonstrated to use less memory and sometimes get higher**

throughput than PyTorch on the same hardware [67] [68] **. Outside of TPUs, JAX works on GPUs (via CUDA) and even CPU (via XLA CPU backend), so it's quite portable. In essence,** JAX represents the new breed of ML frameworks** that rely on compilers (XLA) rather than custom C++ kernels for each operation, giving both flexibility and performance.

- **Triton (OpenAI/META's Deep Learning Compiler):** Triton is another exciting project – essentially a **Python-based DSL (Domain Specific Language)** and compiler for writing custom GPU kernels. OpenAI introduced Triton in 2021 to let researchers write GPU code at a high level (in Python) and have it compile to CUDA kernels that rival the efficiency of hand-tuned code [72] [73] . Triton uses an LLVM-based backend; users write C-like code in a Pythonic way (using decorators), and Triton JIT-compiles it to PTX (NVIDIA's GPU ISA). The advantage is that you can create tailored kernels (for example, a fused operation specific to your model) without diving into CUDA C++ or assembly. Triton has been used to implement high-performance versions of things like layer normalization, matrix multiply, convolution, and even parts of transformer kernels, often outperforming existing library routines by specializing to certain sizes. **Triton integrates with PyTorch** – in fact, PyTorch 2.0 uses a Triton-based implementation for its new 2.0 Attention and other kernels. It has shown particularly strong results with PyTorch 2.0 and is seen as a way to break the dependence on NVIDIA's closed-source cuDNN/BLAS for every new operation [72] [74] . While Triton currently requires a CUDA-enabled GPU (NVIDIA) to run, its design is hardware-agnostic, and there are plans to support other backends (AMD, etc.) in the future [72] . In summary, **Triton empowers developers to optimize at the kernel level** with relative ease – you get CUDA-like performance with Python-like convenience. It's part of a broader trend of *open-source compilers lowering the barrier* to fully exploiting hardware. As evidence of its impact: OpenAI reported that Triton and PyTorch 2.0 together can sometimes *beat* Nvidia's own libraries, and projects like Meta's TorchInductor now use Triton for many ops. This helps reduce the CUDA monopoly by providing an open, flexible alternative [72] .

Performance-per-Watt Trade-offs for LLM Workloads

For large language models, raw speed is not the only concern – **energy efficiency** (performance per watt) is crucial, because training or serving an LLM can consume massive power over days or months. Different hardware choices can lead to significant differences in power usage for the same workload. Here we consider some key factors and findings regarding performance-per-watt:

- **Specialization vs Generality:** Specialized accelerators (TPUs, IPUs, etc.) often achieve better perf/W by cutting out general-purpose overhead and focusing on matrix math efficiency. For instance, Google reported that in a head-to-head comparison on a large-scale LLM training, a TPU v4 system was about *1.2–1.7× faster than an equivalent NVIDIA A100 GPU system while using 1.3–1.9× less power* [75] [76] . That implies a notable performance-per-watt advantage for TPU (roughly 2×, depending on exact comparison). Similarly, Graphcore claims 2–3× higher energy efficiency than GPUs on sustained workloads due to their architecture [12] . The gap largely comes from architectural efficiency (e.g. TPU's systolic array is utilized at high efficiency for dense ops, while GPUs might not hit peak utilization for all workloads, and IPUs minimize memory energy by on-chip storage). However, specialized chips may lose some flexibility – if your model strays from what the hardware is optimized for, efficiency can drop.

- **Memory and Data Movement:** As models grow (2B, 20B, 200B parameters), memory access dominates energy. Chips that can reduce off-chip memory use will have better perf/W. This is why

HBM-equipped GPUs or on-die SRAM in IPUs can improve energy usage despite similar computation specs. A rule of thumb: **computing 1 arithmetic operation might cost <1 picojoule, but a DRAM access can cost ~100s of picojoules**. So skipping unnecessary memory fetches (through caching, reuse, sparsity) yields big power savings. Techniques like quantization also help here (smaller data -> less memory traffic). In LLM inference, often memory bandwidth is a limiter (especially if model doesn't fully fit on chip). Some accelerators like Cerebras WSE avoid the GPU-like external memory entirely, which can give efficiency benefits for models that fit entirely on the wafer (no costly PCIe or networking for data). In distributed training, **interconnect efficiency** matters too – TPU v4 used optical interconnects to reduce communication energy and scale to 4096 chips efficiently [77] [46]. This contributed to Google's result that their TPU v4 pods in a well-designed datacenter use **2–6× less total energy and 20× less carbon footprint** than typical on-prem GPU clusters for similar workloads [76] [78]. Efficient cooling and power delivery in datacenter-scale systems also factor into that.

- **Large Batch vs Low Batch:** LLM workloads vary – training uses large batch sizes (thousands of tokens), whereas online inference might be batch-1 (single prompt). Different hardware excels in different regimes. GPUs, with their flexibility and many threads, do well for smaller batches or irregular workloads (like decoding one token at a time with control flow). TPUs or large systolic arrays prefer big batches to fill the matrix unit. So for **inference** perf/W, one might see GPUs being more efficient at low batch, whereas for **training** or batched inference, the specialized accelerators shine. New techniques like multi-stream execution (running many small inference queries in parallel on a big accelerator) try to bridge this.

- **Dynamic Sparsity and Utilization:** LLMs often have parts that are "activated" conditionally (e.g. MoE experts, or steps that depend on sequence length). If hardware can power-gate or skip unused parts, it saves energy. If not, you might be wasting power on idle units. So, an accelerator with fine-grained power management and the ability to exploit model sparsity can improve perf/W. For example, mixture-of-experts models route each input to only a subset of parameters – in theory this reduces FLOPs per inference, but only if the hardware and software can efficiently skip the inactive parts. GPUs can do this at a high level (by not launching kernels for the pruned parts), but some overhead remains. Newer accelerators might include *power gating* for blocks of compute so that, say, if 50% of weights are zero (pruned), those multiplier units simply don't switch and thus save energy. This requires careful design to ensure the overhead of checking for zeros doesn't outweigh the savings.

- **Real-World Efficiency Numbers:** To ground this discussion, consider some real performance/power figures from literature. NVIDIA's A100 GPU (40GB) has a TDP around 400W and delivers ~250–300 TFLOPS FP16. In practice, running a 2B parameter transformer, you might see on the order of a few thousand sequences per second throughput on one A100 for inference (batch size dependent), with ~200–300W power draw (this is a rough estimate; actual numbers vary by model and sequence length). Google's TPU v4 chip (each around 175W TDP for the MXU and with liquid cooling) can similarly deliver several hundred TFLOPS and when used in pods achieves high utilization (~50-60% of peak sustained [7]). The result was that one TPU v4 chip often provides slightly higher throughput than one A100 at lower power – for instance, *TPU v4 was reported ~1.7× faster than A100 while using ~30–50% less power* in a head-to-head MLPerf-type comparison [75]. Another data point: Graphcore's IPU systems running a sparse transformer model could achieve **>100 TFLOPS effective** with only ~150W per chip, due to not using all weights every time (Graphcore's marketing suggests >200

TeraOPS/W on some workloads). Meanwhile, neuromorphic approaches might achieve even higher energy efficiency (e.g. 10^4 spikes per joule), but they cannot directly execute a dense LLM without model conversion. Lastly, the optical computing study estimated that an optical accelerator could perform 1e15 (a quadrillion) MACs with ~1/100th the energy of today's electronics [29] – if that materializes, it could run a 1 trillion parameter model with the energy of a 10 billion parameter model today. But as of now, mainstream accelerators like GPUs/TPUs deliver on the order of **10–30 TFLOPS per 100W** for typical LLM computations (this is 100–300 GFLOPS/W), whereas special systems (low precision or analog) might reach 1000+ GFLOPS/W on specific tasks. The trend is upward: each generation improves perf/W (TPU v4 > TPU v3 by 2.7× [5], NVIDIA Hopper H100 claims ~2× perf/W vs A100, etc.). Techniques like advanced cooling (submersion, etc.) and better voltage scaling also help. Ultimately, **performance-per-watt for LLMs** is as important as raw speed, because models like GPT-3 or GPT-4 can require *megawatt-hours* of energy to train. Hardware choices and optimizations that double efficiency can save significant cost and environmental impact.

Key Takeaway: Selecting hardware for LLMs often involves a trade-off: GPUs offer maturity and versatility, while specialized accelerators (TPUs, IPUs, etc.) can provide superior efficiency and throughput for well-structured large workloads [75]. Achieving the best perf/W also demands leveraging architectural features (tensor cores, sparsity) and software optimizations (compilers, mixed precision). As models continue to grow, this balance will be ever more crucial, and emerging tech like photonics or neuromorphic computing may play a role in the future to keep energy demands in check.

Benchmark Lab – Comparing a 2-Billion-Parameter Transformer on Multiple Backends

In this lab, we will **profile a 2-billion parameter Transformer model** on three different hardware backends and compare performance metrics: FLOPs utilization, latency, and energy consumption. This practical exercise solidifies concepts from the survey by showing how different accelerators and software stacks handle the same workload. We will use a Hugging Face implementation of a GPT-2 style Transformer and run inference on: (1) an NVIDIA GPU with CUDA, (2) an AMD GPU with ROCm, and (3) a Google TPU (via XLA/JAX). You can substitute hardware as available (e.g. if no TPU, using JAX on a GPU with XLA). The goal is to record **throughput (tokens/sec or inference latency)** and **energy usage (Joules)** for each, and compute derived metrics like **FLOPs per token** and **energy per inference**.

Lab Setup:

- **Model:** Use the 2-billion parameter variant of GPT-2 (for instance, EleutherAI's GPT-neo-2.7B or a trimmed-down GPT model around 2B). This ensures the model is large enough to stress the hardware but (hopefully) fits in GPU memory. Using HuggingFace Transformers, you can load the model and a sample text input for inference.
- **Backends:** Prepare three environments:
- **PyTorch+CUDA (NVIDIA GPU)** – e.g. an A100 or V100 if available. Ensure `transformers` is using the GPU (`.to('cuda')`).
- **PyTorch+ROCm or TensorFlow+ROCm (AMD GPU)** – requires an AMD GPU like MI50/MI100 or newer, and a properly installed ROCm stack. Alternatively, if ROCm is not available, you could use an AMD instance on cloud or skip this if necessary.

- **JAX+TPU (Google Cloud TPU)** – if you have access to a TPU v3-8 or v4-8 via Google's TPU Research Cloud or Cloud TPU service. Install `jax` and `flax` and use the TPU as the device. If a physical TPU is not accessible, you can simulate XLA on CPU/GPU, but for true comparison a TPU is ideal.

- **Energy Measurement:** Use hardware counters where possible. On CPU servers, one can use Intel RAPL interfaces or AMD's energy counters to measure socket energy. For GPUs and TPUs, measuring energy is trickier:

- On NVIDIA GPUs, you can poll `nvidia-smi` for power usage (Watts) during the run. NVIDIA's NVML library or PyTorch's built-in profiler can also report energy consumption on some systems.
- On AMD, use `rocm-smi` to get power usage.
- On Google TPUs, energy is not directly exposed, but you might use cloud monitoring if available. If not, we can approximate using TDP if utilization is known (e.g. assume TPU v3-8 = 8× (TPU TDP ~200W) when fully utilized).

- **Important:** Ensure to run a warm-up and then measure a steady-state during inference of a large batch or many tokens to get consistent readings.

- **Procedure:** For each backend, run a forward pass of the model on a sample input (or multiple iterations) and time it. Use high precision timing (e.g. `torch.cuda.Event` for GPU, or `time.time()` around the code, etc.). Simultaneously, record power if possible (start a background process logging power at 100ms intervals). It's helpful to run a relatively long inference or many repetitions to measure energy (because instantaneous power might fluctuate, but energy over a second or two is more stable).

Measurements to Record:

1. **Latency per Token:** How long (in milliseconds) it takes to generate one token (or to do one forward pass for a given sequence length). For instance, you might measure that on the GPU it takes 50 ms to produce one output token from the model (depending on batch size and sequence).
2. **Throughput (tokens/second):** This is just the inverse of latency if doing one token at a time, or if doing batch inference, how many tokens can be processed in parallel per second. For a fair comparison, you might use batch size = 1 or a fixed small batch for all. Alternatively, measure at a reasonable batch that saturates each device (to see max throughput).
3. **FLOPs Utilization:** Calculate the theoretical number of floating-point operations the model requires for one forward pass. A rough estimate: Each transformer layer involves matrix multiplies on the order of O(n * d * d) where n is sequence length and d model dim, plus attention (which is O(n^2 * d) for self-attention). For a simple estimate, use 2 * (number of model parameters) as the number of FLOPs for one token output (this assumes each weight is used roughly once for a token, which is an approximation). For a 2B model, that's ~4 billion FLOPs per token. Now compare that to hardware peak FLOPS (e.g. A100 ~312 TFLOPS FP16). If you got 50 ms per token on A100, then you achieved ~4e9 FLOPs / 0.05 s = 80e9 FLOPs/s = 80 GFLOPS, which is only 0.025% of peak 312e12! (This implies a lot of time is spent waiting for data, etc., which is expected for batch-1 inference). For a more apples-to-apples, you could use batch such that you approach higher utilization. But reporting this utilization is insightful – it shows how hardware utilization drops in low-batch scenarios.
4. **Power/Energy:** Note the average power draw during the run and the total energy for processing a fixed number of tokens. For example, if the GPU was at ~250W and it took 0.05 s per token, that's

about 12.5 J per token. On a TPU that might have, say, 100W per chip active and processed the token in 0.03 s, that's 3 J per token – significantly less. These are hypothetical, but you should compute whatever your measurements show. Energy per token or per sequence is a nice comparative metric for efficiency.

Example Results Format:

After running the tests, tabulate or list the results for each backend. For instance:

- **NVIDIA A100 (CUDA, batch=1):** Latency ~50 ms/token (20 tokens/sec). ~4e9 FLOPs per token => ~80 GFLOPs/sec achieved. Power ~250 W, so ~12.5 J per token. (Perf/W: ~1.6 billion FLOPs/Joule).
- **AMD MI100 (ROCm, batch=1):** Latency ~60 ms/token (16 tokens/sec). Achieved ~66 GFLOPs/sec. Power ~300 W (hypothetical), so ~18.7 J per token. (Perf/W: ~0.9 billion FLOPs/J). *Note:* AMD's higher latency might be due to slightly lower clock or less optimized kernel in this scenario – this is an illustrative example.
- **Google TPU v3-8 (XLA/JAX, batch=1):** Latency ~30 ms/token (33 tokens/sec). Achieved ~133 GFLOPs/sec. TPU power ~120 W (for 8 cores of one TPU device, each ~15W if not fully utilized), so ~3.6 J per token. (Perf/W: ~3.7 billion FLOPs/J).

(Disclaimer: The above numbers are only illustrative to show format; actual results will vary based on model, sequence length, and system.)

From such results, you can discuss why differences occur. In the hypothetical example, the TPU was more efficient for this single-token inference because its architecture handled the model well and XLA optimized the execution, whereas the GPU was underutilized (likely waiting on memory). The AMD GPU might lag due to less optimized software or lower boost clocks. If you increase batch size, you'd expect all devices to improve throughput (and utilization). You could try batch=8 or 16 and see if NVIDIA/AMD GPUs catch up on utilization. Often, NVIDIA GPUs excel when they can fill their SMs with work (so larger batch or sequence), whereas TPUs need larger batches to avoid underutilization of the matrix units but have excellent inter-core communication for scaling.

Analysis Questions:

After collecting data, reflect on: - Which backend delivered the highest throughput? Was it also the best in energy per output?
- How close to theoretical peak FLOPs did each get? (This indicates how well the software stack and model mapping kept the hardware busy.)
- What might be the bottlenecks in each case (compute bound vs memory bound)? For instance, the GPU might be memory-bandwidth bound for this model, whereas the TPU's systolic array might be fully occupied.
- How did the software frameworks impact results? (e.g. JAX+XLA might have fused ops better than PyTorch eager mode. PyTorch with CUDA is highly optimized by libraries, whereas on ROCm some kernels might not be as mature, affecting performance.)
- If you have data for a larger batch or different sequence lengths, how do the trends change? (This can show at what point each device hits its stride.)

By performing this lab, you get a **firsthand sense of the performance-per-watt trade-offs**. It complements the earlier discussion: hardware architecture is one factor, but software and usage pattern

(batching, precision, etc.) also determine efficiency. The lab underscores why, for example, **NVIDIA GPUs remain popular** – their absolute performance is very high, but it also shows the potential gains from specialized hardware like TPUs given the right conditions (especially in energy efficiency, as seen in the example where TPU used far fewer joules per token). These kinds of empirical comparisons are essential for making informed decisions about deploying LLMs in practice, where both speed and power cost matter.

Conclusion

The landscape of AI hardware accelerators is rapidly evolving. We started with GPUs enabling the deep learning revolution in 2012, and since then a plethora of specialized chips have emerged to push the boundaries of performance and efficiency [1] [2]. Today's state-of-the-art includes **GPUs** with thousands of cores and tensor units, **TPUs** with massive systolic arrays, **IPUs** and ASICs with innovative memory-centric designs, and experimental **neuromorphic** and **optical** processors that chart the future of low-power AI. Key architectural concepts like on-chip memory, dataflow optimization, low-precision computation, and sparsity exploitation are common themes across these platforms, each contributing to better performance-per-watt for demanding workloads [54] [55].

Software stacks play an equally important role – frameworks like CUDA and ROCm give developers access to GPU power, compilers like XLA and TVM maximize hardware utilization, and new languages like Triton lower the barrier to optimizing kernels [72] [73]. For practitioners, it's crucial to stay informed about these tools to fully leverage the available hardware.

Large language models, in particular, stress current hardware to the limit, and we've seen that specialized accelerators (TPUs, etc.) can offer significant efficiency gains [75]. However, no single solution fits all: GPUs provide flexibility and maturity, while dedicated chips can excel for specific workloads at scale. The **performance-per-watt trade-off** will continue to be a key consideration, especially as model sizes grow faster than chip TDPs can.

In our benchmark lab, we put theory into practice by measuring a 2B-param Transformer on different backends. Such hands-on evaluation is invaluable – it reveals the real-world gaps between peak specs and achieved performance, and highlights how much power is actually used to generate those clever outputs from an LLM. These insights will guide optimizations (maybe we need to increase batch size, or use quantization, or try a different compiler) and future hardware selection (if energy is a concern, perhaps consider that TPU or an upcoming low-power accelerator).

In conclusion, the synergy of advanced hardware designs and smart software is what enables today's AI at scale. As you continue exploring this field, keep an eye on emerging architectures (like photonic chips or next-gen TPUs) and evolving software ecosystems (open compiler initiatives, etc.). The ultimate goal is clear: achieve human-level AI capabilities with efficient, affordable computation. The progress in **AI hardware accelerators** surveyed here gives plenty of reason for optimism, as each new generation brings us closer to that goal with improved speed and efficiency [5] [76].

References: (Hardware Surveys & Key Papers)
1. *A Survey of Deep Learning Hardware Accelerators*, IEEE Computer Surveys, 2023 – Comprehensive overview of GPU, TPU, FPGA, ASIC, NPU, RISC-V, PIM, neuromorphic, photonic, etc., including taxonomy and future trends [79] [80] [4].
2. *From GPUs to TPUs: The Evolution of AI Compute*, Comm. ACM, 2022 – Discussion of how AI workloads drove

a shift from general-purpose GPUs to domain-specific chips, and the competitive landscape (Nvidia vs Google vs others) [8] [81].
3. *Sparsity-Aware Accelerators for LLMs*, MLSys 2024 – Latest research on leveraging model sparsity (pruning, MoEs) in hardware. E.g. NVIDIA Ampere's 2:4 sparse tensor core support and novel accelerator designs for sparse Transformers [42] [44].
4. *Optical Transformers: Early Results*, Nature Photonics 2024 – Demonstration that optical computing can run Transformer models with potential 100× efficiency gains, discussing prototypes and energy scaling for photonic matrix multiplies [29] [82].

---

[1] [2] [4] [5] [10] [11] [12] [13] [14] [15] [16] [17] [18] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [79] [80] [2306.15552] A Survey on Deep Learning Hardware Accelerators for Heterogeneous HPC Platforms
https://ar5iv.labs.arxiv.org/html/2306.15552

[3] [8] [9] [63] [64] [65] [66] [72] [73] [74] [81] Nvidia at the Center of the Generative AI Ecosystem—For Now – Communications of the ACM
https://cacm.acm.org/opinion/nvidia-at-the-center-of-the-generative-ai-ecosystem-for-now/

[6] [7] [45] [46] [75] [76] [77] [78] ISCA 2023 Camera Ready TPUv4 Paper
https://arxiv.org/pdf/2304.01433

[19] [20] [21] [22] [23] [24] [31] [32] [33] An integrated large-scale photonic accelerator with ultralow latency | Nature
https://www.nature.com/articles/s41586-025-08786-6?error=cookies_not_supported&code=bf33fe69-0693-4842-b5a4-329b012f9eb3

[25] [26] [27] [28] [29] [30] [82] Optical Transformers | OpenReview
https://openreview.net/forum?id=Xxw0edFFQC

[67] [68] Echo Nolan's Blog - JAX vs PyTorch: A simple transformer benchmark
https://www.echonolan.net/posts/2021-09-06-JAX-vs-PyTorch-A-Transformer-Benchmark.html

[69] [70] [71] [1802.04799] TVM: An Automated End-to-End Optimizing Compiler for Deep Learning
https://arxiv.org/abs/1802.04799